

使用 Rust 编写转发 DLL

本文实现一个转发 DLL： `version.dll` 转发到系统的 `version.dll` 上。另外要注意，本文代码仅供参考，不一定可以运行，需要做一定的修改，或直接查看最终实现：

<https://github.com/hamflx/forward-dll>。

首先用下面的命令查看系统的 `version.dll` 导出函数：

```
C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise>dumpbin /exports
c:\windows\system32\version.dll
Microsoft (R) COFF/PE Dumper Version 14.29.30136.0
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Dump of file c:\windows\system32\version.dll
```

```
File Type: DLL
```

```
Section contains the following exports for VERSION.dll
```

```
00000000 characteristics
927B71E6 time date stamp
    0.00 version
        1 ordinal base
        17 number of functions
        17 number of names
```

ordinal	hint	RVA	name
1	0	00001080	GetFileVersionInfoA
2	1	00002190	GetFileVersionInfoByHandle
3	2	00001DF0	GetFileVersionInfoExA
4	3	00001040	GetFileVersionInfoExW
5	4	00001010	GetFileVersionInfoSizeA
6	5	00001E00	GetFileVersionInfoSizeExA
7	6	00001050	GetFileVersionInfoSizeExW
8	7	00001060	GetFileVersionInfoSizeW
9	8	00001070	GetFileVersionInfoW
10	9	00001E10	VerFindFileA
11	A	00002360	VerFindFileW
12	B	00001E20	VerInstallFileA
13	C	00002F80	VerInstallFileW
14	D		VerLanguageNameA (forwarded to KERNEL32.VerLanguageNameA)
15	E		VerLanguageNameW (forwarded to KERNEL32.VerLanguageNameW)
16	F	00001020	VerQueryValueA
17	10	00001030	VerQueryValueW

```
Summary
```

```
1000 .data
1000 .pdata
2000 .rdata
1000 .reloc
1000 .rsrc
3000 .text
```

```
C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise>
```

然后我们拿 `GetFileVersionInfoSizeA` 举例来写一个实例函数：

```
static mut RealGetFileVersionInfoSizeA: usize = 0;

#[no_mangle]
pub extern "system" fn GetFileVersionInfoSizeA() -> u32 {
    unsafe {
        std::arch::asm!(
            "jmp rax",
            in("rax") RealGetFileVersionInfoSizeA,
            options(nostack)
        );
    }
    1
}
```

这个函数其实也不能叫函数，因为它不会返回，直接跳转到目标函数地址，这个目标函数地址需要在 `DllMain` 中通过 `LoadLibrary` 和 `GetProcAddress` 进行赋值：

```
#[no_mangle]
pub extern "system" fn DllMain(_inst: isize, reason: u32, _: *const u8) -> u32
{
    if reason == 1 {
        let version_module =
            load_library("c:\\windows\\system32\\version.dll");
        unsafe { RealGetFileVersionInfoSizeA = get_proc_address(version_module,
            "GetFileVersionInfoSizeA") };
    }
    1
}
```

如果每个函数都这么写，那是相当的麻烦，因此，我们可以写一个宏，并把加载目标 dll 的真实地址封装到结构体的方法里面，这样在 `DllMain` 时直接调用即可：

```
#[macro_export]
macro_rules! forward_dll {
    ($lib:expr, $name:ident, $($proc:ident)* ) => {
        static mut $name: forward_dll::DllForwarder<{ forward_dll::count!
        ($($proc)* )> = forward_dll::DllForwarder {
            lib_name: $lib,
            target_functions_address: [
                0;
                forward_dll::count!($($proc)*
            ],
            target_function_names: [
                $(stringify!($proc),)*
            ]
        };
        forward_dll::define_function!($name, 0, $($proc)*);
    };
}
```

```
#[macro_export]
macro_rules! define_function {
    ($name:ident, $index:expr, ) => {};
    ($name:ident, $index:expr, $proc:ident $($procs:ident)* ) => {
        #[no_mangle]
        pub extern "system" fn $proc() -> u32 {
            unsafe {
                std::arch::asm!(
                    "jmp rax",
                    in("rax") $name.target_functions_address[$index],
                    options(nostack)
                );
            }
            1
        }
        forward_dll::define_function!($name, ($index + 1), $($procs)*);
    };
}
```

/// DLL 转发类型的具体实现。该类型不要自己实例化，应调用 forward_dll 宏生成具体的实例。

```
pub struct DllForwarder<const N: usize> {
    pub target_functions_address: [usize; N],
    pub target_function_names: [&'static str; N],
    pub lib_name: &'static str,
}

impl<const N: usize> DllForwarder<N> {
    /// 将所有函数的跳转地址设置为对应的 DLL 的同名函数地址。
    pub fn forward_all(&mut self) -> ForwardResult<()> {
        let load_module_dir = "C:\\Windows\\System32\\";
        let module_full_path = format!("{}", load_module_dir, self.lib_name);
        let module_handle = get_module_handle(module_full_path.as_str())?;

        for index in 0..self.target_functions_address.len() {
            let addr_in_remote_module =
                get_proc_address_by_module(module_handle,
                self.target_function_names[index]);
            self.target_functions_address[index] = addr_in_remote_module as
            *const usize as usize;
        }
    }
}
```

```
    }

    Ok(())
}

forward_dll::forward_dll!(
    "C:\\\\Windows\\\\system32\\\\version.dll",
    DLL_VERSION_FORWARDER,
    GetFileVersionInfoA
    GetFileVersionInfoByHandle
    GetFileVersionInfoExA
    GetFileVersionInfoExW
    GetFileVersionInfoSizeA
    GetFileVersionInfoSizeExA
    GetFileVersionInfoSizeExW
    GetFileVersionInfoSizeW
    GetFileVersionInfoW
    VerFindFileA
    VerFindFileW
    VerInstallFileA
    VerInstallFileW
    VerLanguageNameA
    VerLanguageNameW
    VerQueryValueA
    VerQueryValueW
);

// 在 DllMain 中调用:
// unsafe { DLL_VERSION_FORWARDER.forward_all() };
```

这就完成了 `version.dll` 的转发。可参考通过该方法实现的一个小工具：

<https://github.com/hamflx/huawei-pc-manager-bootstrap>。

法二

如果我们不希望通过 `DllMain` 来初始化怎么办？我们可以在跳板函数里面加载目标函数地址，为了保证寄存器和栈上数据的状态，我们单独写一个加载函数，并从跳板里面调用过去，由编译器来帮我们保证寄存器的状态。

通过该函数拿到目标函数地址后将其返回，那么目标函数地址就存储在 `rax` 上，然后再跳转到 `rax` 上：

```

pub extern "system" fn $proc() -> u32 {
    unsafe {
        std::arch::asm!(
            "push rcx",
            "push rdx",
            "push r8",
            "push r9",
            "push r10",
            "push r11",
            options(nostack)
        );
        std::arch::asm!(
            "sub rsp, 28h",
            "call rax",
            "add rsp, 28h",
            in("rax") forward_dll::default_jumper,
            in("rcx") std::concat!($lib, "\\0").as_ptr() as usize,
            in("rdx") std::concat!(std::stringify!($proc), "\\0").as_ptr() as
usize,
            options(nostack)
        );
        std::arch::asm!(
            "pop r11",
            "pop r10",
            "pop r9",
            "pop r8",
            "pop rdx",
            "pop rcx",
            "jmp rax",
            options(nostack)
        );
    }
    1
}

```

然后我们实现一个 `forward_dll::default_jumper` 方法：

```

/// 默认的跳板，如果没有执行初始化操作，则进入该函数。
pub fn default_jumper(
    lib_name: *const u8,
    func_name: *const u8,
) -> usize {
    let module_handle = unsafe { LoadLibraryA(lib_name) };
    if module_handle != 0 {
        let addr = unsafe { GetProcAddress(module_handle, func_name) };
        // 这里调用了 FreeLibrary 释放目标模块，实际使用需要在其他地方持有目标模块的句柄，
        防止被释放。
        unsafe { FreeLibrary(module_handle) };
        return addr.map(|addr| addr as usize).unwrap_or(exit_fn as usize);
    }

    exit_fn as usize
}

```

0条评论

hamflx ▼



开始讨论...



分享

最佳 最新 最早

来做第一个留言的人吧!

订阅

隐私

不要出售我的数据